Attention-Knockout Reasoning Benchmarks Reproducibility Specification

`benchmarks/entropy_exp.py` — De Bruijn experiments

Generated 2026-09-04 | git `e4c1e6e` |
https://github.com/amrutn/debruijn_experiments

1 Overview

A single driver, `benchmarks/entropy_exp.py`, runs three teacher-forced / generation analyses of Qwen3 models in which the model's attention is *knocked out* so that each query token may attend only to the prompt plus the n most recent tokens (a sliding "memory window" with the true positional encodings preserved). The three experiments are:

1. **knockout** — perplexity of ground-truth reasoning traces vs. the memory-window size n (teacher forcing).
2. **knockout-generation** — the model *generates* answers under the same knockout; accuracy and generated-reasoning length vs. n.
3. **entropy** — mean per-token Shannon entropy vs. answer-token position (control).

The experiment run is selected with `--experiment knockout,knockout-generation,entropy,all` (default `all`). Everything below is the exact configuration required to reproduce the results; all knobs are command-line flags with the defaults listed.

2 Models

Role	Hugging Face id	Notes
Primary	`Qwen/Qwen3-14B`	default `--model`
Comparison	`Qwen/Qwen3-32B`	`--compare-model` (dashed overlay)

- Precision: `float16` on CUDA (`float32` on CPU/MPS).
- Chat template applied with `add_generation_prompt=True` and **thinking mode on** (`enable_thinking=True`).
- Tokenizer: `AutoTokenizer.from_pretrained(<model>)`; EOS token set is {`eos_token_id`, `<|im_end|>`}.
- Caches are namespaced per model (`knockout_gen/<slug>/`), so 14B and 32B never collide and either can be re-run independently.

3 Datasets

Benchmark	HF id / config	Split	Notes
GSM8K	`openai/gsm8k` / `main`	`test` (gen), `train` (ppl/entropy)	grade-school math
MATH-500	`HuggingFaceH4/MATH-500`	`test`	competition math
GPQA-Diamond	`Idavidrein/gpqa` / `gpqa_diamond`	`train` (198 items)	**gated**: requires HF auth + accepted terms; skipped with a warning if unauthenticated
WikiText-2	`Salesforce/wikitext` / `wikitext-2-raw-v1`	`train`	non-reasoning baseline (perplexity/entropy only)

Problem sampling (generation). `--gen-samples 192` problems per benchmark, drawn as a *stable seeded-permutation prefix*:

```
idx = numpy.random.default_rng(seed).permutation(N)[:min(n_samples, N)] # seed = --gen-seed = 42
```

so a larger `--gen-samples` is a strict superset of a smaller one (rows are reusable). GPQA has 198 diamond items ⇒ 192 used; its four options are shuffled with a per-problem stable seed `default_rng([42, i])`, and the gold answer is recorded as the resulting letter (A–D).

4 The attention-knockout mechanism

At memory-window size n, a query at absolute position q may attend to key k iff it is causal *and* (k is in the prompt *or* within the last n tokens):

$$\text{allowed}(q,k) \;=\; (k \le q) \;\wedge\; \big(k < P \;\vee\; k > q - n\big),$$

where P is the prompt length. **True (absolute) position ids are always used**, so dropping the middle of the context leaves the same rotary-position-embedding (RoPE) gap the tokens would have had — the model sees a prompt "sink" plus a recent window at their real positions.

- **Perplexity (teacher forcing).** A custom 4-D additive mask ($0/-\infty$) implements the rule above via SDPA. Per-token NLL is taken over the answer region (positions P:L), predicting token t from the logits at $t-1$; sequence perplexity = $\exp(\overline{\text{NLL}})$. No query is ever fully masked (all see the prompt), so no NaN guard is needed.
- **Generation.** A rolling-window KV cache: the prompt KV is prefilled once and kept as a sink; each step appends the new token's KV and evicts the oldest so the cache holds `[prompt] + [last n-1]` tokens (the query then attends to $n-1$ cached + itself = the n most recent). Explicit true `position_ids` preserve the RoPE gap. Decode is O(tokens).
- **Full-context reference.** The sentinel window `n = GEN_FULL_N = 100000` disables eviction, giving ordinary full-causal generation — plotted as the "full" point.

5 Decoding / generation settings

Setting	Flag	Default
Sampling	(nucleus)	top-p + temperature
Temperature	`--gen-temperature`	`0.6`
Top-p (nucleus)	`--gen-top-p`	`0.95`
Max new tokens	`--gen-max-new-tokens`	`12800`
Seed (sampling + selection)	`--gen-seed`	`42`
Problems / benchmark	`--gen-samples`	`192`
Batch size (window cols)	`--gen-batch-size`	`128`
Batch size (full col)	`--gen-full-batch-size`	`16`

Sampling uses a custom `sample_top_p` (temperature scaling then nucleus filtering). Batch sizes affect only speed/memory, never outputs, and are *not* part of the cache identity. The window/full split exists because the full column's KV grows with generation length while the window columns are bounded at `prompt + n`.

6 Prompt templates (verbatim)

Each user message is `"<problem>\n\n<instruction>"`, wrapped by the model's chat template (thinking on). Instructions:

```
GSM8K : "Please reason step by step. End your response with '#### ' followed by the
    final numeric answer."
MATH : "Please reason step by step, and put your final answer within \boxed{}."
GPQA : "<question>\n\n(A) ...\n(B) ...\n(C) ...\n(D) ...\n\nReason step by step, then
    give the letter of the correct option within \boxed{}."
```

7 Answer grading

- **GSM8K** (`kind='gsm8k'`): take text after the last `####` (else whole text), extract the last number, compare to gold as float within `1e-6` (fallback: string equality). Gold = the dataset's post-`####` value with commas stripped.
- **MATH-500** (`kind='math'`): `math_verify.parse`/`verify` (semantics-aware) on the generation vs. `\boxed{gold}`; if `math_verify` is unavailable, fall back to a normalized `\boxed{}` string match.
- **GPQA** (`kind='mcq'`): extract the last `\boxed{...}` letter (A–D) and require exact match to the shuffled gold letter.

8 Memory-window sweep

Experiment	n values (+ *full* where applicable)
knockout-generation	`20,30,40,50,60,70,80,90,100,120,140,160,180,200,400,500,600,700,800,900,1000` + full
knockout (perplexity)	`5,10,15,20,30,40,50,60,70,80,90,100` + full

Set via `--knockout-gen-ns` and `--knockout-ns`. Sweep columns are reused per-n: adding a new n computes only that column.

9 Other experiment knobs

Setting	Flag / constant	Default
Min answer length (knockout ppl)	`--min-answer-len`	`100` tokens
Entropy analysis length	`--max-analysis-len`	`100` tokens
Entropy cache min length	`--cache-min-len`	`32` tokens
Forward-pass length cap	`MAX_SEQ_LEN`	`1024`
WikiText passages considered	`NUM_WIKI_SAMPLES`	`5000`
WikiText "prompt" length	(override)	mean GSM8K prompt length
Per-dataset cap (optional)	`--num-samples`	`None` (all)

10 Device placement & memory management

- `--devices cuda:0 cuda:1 ...` sets `CUDA_VISIBLE_DEVICES`; the model is placed with `device_map="auto"`.
- Weights are spread *evenly* across the visible GPUs using accelerate's `get_balanced_memory` (with `low_zero=True`); a plain `device_map="auto"` or a per-GPU cap otherwise packs the model onto the low-index GPUs and OOMs on the batched prefill.
- `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` is set to reduce allocator fragmentation.
- Prefill uses `logits_to_keep=1` to avoid materializing the `(batch, prompt_len, vocab)` logits tensor.

11 Caching & resumability

- Generation results: `knockout_gen/<model-slug>/<benchmark>.npz` holding the per-(problem,n) `correct` matrix (-1 = uncomputed), the matching `lengths` matrix, the `n_values`, and a `meta` dict. A `length_summary.csv` is also written.
- Perplexity: `knockout_ppl/<slug>/<benchmark>.npz` (+ `__full`); raw entropy traces: `entropy_traces/<slug>/<benchmark>.npz`.
- **Cache identity** (what forces recompute): `model`, `gen_seed`, `gen_max_new_tokens`, `gen_temperature`, `gen_top_p`. It deliberately excludes `gen_samples` and batch sizes (stable-prefix sampling makes larger runs reuse existing rows).
- Runs are resumable per cell (checkpointed every batch); `--plot-only` re-aggregates figures from cache with no model load; `--force` recomputes.

12 Software environment

Two conda environments are provided at the repo root:

- `environment.yml` — macOS (MPS) or CUDA 12.x; torch cu126 wheel note.

- `environment2.yml` — CUDA 13.0 / single H100; `torch>=2.13` (cu130), env name `ngram-experiments-cu130`.

Key pinned packages (both): `python=3.12`, `numpy>=2.0`, `scipy>=1.11`, `matplotlib>=3.8`, `transformers>=4.56` ($\geq$4.51 for Qwen3; $\geq$4.56 for the `dtype=` kwarg), `datasets>=2.19`, `accelerate>=0.26`, `peft>=0.11`, `sympy>=1.12`. `math_verify` is used for MATH grading if importable (optional). *Note:* newer `huggingface_hub` requires namespaced dataset ids (hence `openai/gsm8k`, `Salesforce/wikitext`).

13 Hardware configurations

Config	GPUs	Suggested batch
Cluster (original)	6× RTX 4090 24 GB	`--gen-batch-size 48 --gen-full-batch-size 4`
Single H100 (14B)	1× H100 80 GB	defaults (`128`/`16`)
Single H100 (32B)	1× H100 80 GB	`--gen-batch-size 32 --gen-full-batch-size 4`

On multi-GPU 4090s, `device_map` gives naive pipeline parallelism (one GPU active at a time). A single H100 holds either model on one device.

14 Commands to reproduce

```
# GPQA is gated: authenticate once (accept terms at hf.co/datasets/Idavidrein/gpqa)
export HF_TOKEN=hf_... # or: huggingface-cli login

# --- Single H100 (recommended) ---
# 14B, all three experiments:
python entropy_exp.py --experiment all --model Qwen/Qwen3-14B
# 32B, generation only (smaller batches -- 64 GB of weights):
python entropy_exp.py --experiment knockout-generation --model Qwen/Qwen3-32B \
    --gen-batch-size 32 --gen-full-batch-size 4

# --- 6x RTX 4090 cluster ---
python entropy_exp.py --experiment all --model Qwen/Qwen3-14B \
    --devices cuda:0 cuda:1 cuda:2 cuda:3 cuda:4 cuda:5 \
    --gen-batch-size 48 --gen-full-batch-size 4

# --- Overlay 32B (dashed) on 14B (solid); no GPU, reads both caches ---
python entropy_exp.py --experiment knockout-generation --plot-only \
    --model Qwen/Qwen3-14B --compare-model Qwen/Qwen3-32B
```

Figures are written to `figures/`; tables and the 75%-of-full-accuracy memory sizes are printed to stdout.

15 Provenance of the current cached results

Recorded from the `.npz` metadata at generation time (seed 42, cap 12800, temperature 0.6, top-p 0.95, 192 problems):

Model	Benchmarks	n completed
`Qwen3-14B`	GSM8K, MATH-500, GPQA	all (20–1000) + full, 192 problems
`Qwen3-32B`	GSM8K only	20–200 (no $n \geq 400$, no full; partial run)

Repository `amrutn/debruijn_experiments`, commit `e4c1e6e` (2026-09-04). Driver: `benchmarks/entropy_exp.py`.